PROGRAM REVIEW / UPDATED 11 OCTOBER 2026

QUEUE2PLAY

Current functionality, DSA and review status

The main rental workflows pass the current regression suite. The two presentation findings from the earlier review have been fixed. F1 and F2 remain documented and are retained intentionally at the owner's request.

Owner clarification: Restoring built-in account passwords (F1) and keeping the original legacy-account backup (F2) are intentional behaviors. They are not requested fixes. Their original observations and practical effects remain in this report.

Verification	Latest observed result
Application sources and regression tests	Compilation passed with Java JDK 24.
Full behavior suite	43,586 checks passed.
Colored / ASCII layout suites	42,835 / 42,833 checks passed.
Real login cooldown	Dedicated check passed; 30 seconds measured.
Supplementary documentation checks	16 assertions passed for inventory, membership and lists.

What the application does

A Java terminal application for PS5, Xbox and Nintendo rentals. Customers register, log in, browse games, build orders, queue requests and view history. Administrators approve eligible requests, process returns, manage inventory and inspect audit records.

Status of the original findings

ID	Original priority	Current status
F1	High	Intentional: startup restores built-in password defaults.
F2	High	Intentional: the first legacy backup can retain plaintext.
F3	Medium	Fixed: large receipts, batch previews and return results are paged.
F4	Medium	Fixed: receipt waiting counts use predicted shared-stock readiness.

The fixes changed presentation and navigation only. Queue eligibility, stock, VIP fairness, prices and rental processing rules remain unchanged. This is automated verification plus source review, not a guarantee for every possible input.

01 / Program structure

Queue2Play coordinates the UI, accounts, catalog, queue, rentals and storage. Main remains the executable entry point. The application has 16 top-level Java source files.

File	Current responsibility
Main.java	Creates Queue2Play, starts it and reports input, storage or validation failures.
Queue2Play.java	Customer/admin menus, cart flow, checkout sessions, returns and inventory actions.
TerminalUI.java	Terminal input, headings, tables, choice panels, slips, messages, confirmations and responsive pages.
AccountManager.java	Registration validation, identifier lookup, login attempts, cooldown and startup account setup.
GameCatalog.java	Game IDs, inventory validation, platform filtering, linear search and insertion sorting.
CheckoutQueue.java	Custom linked queue, VIP grouping, stock eligibility, fairness and predicted positions.
RentalManager.java	Pricing, cart/checkout checks, active rentals, due-date order, late fees and stock restoration.
RentalDataStore.java	File parsing/validation, serialization and restoration of in-memory snapshots after failed saves.
AppStorage.java	File locking, external-change checks, backups and recoverable multi-file journal commits.
PasswordSecurity.java	Salted PBKDF2 password hashing and verification.
Game / User / Rental / RentalRequest	Domain objects holding inventory, account, rental and queued-request data.
StorageException / InputEndedException	Error types for failed storage operations and ended input.
run.ps1	Compiles sources and supplies UTF-8, terminal width/height and the project data directory.

Saved files

accounts.txt, games.txt, rentals.txt, queue.txt and logs.txt are saved together. Rentals retain amounts, quantities, dates and return details. Queue metadata retains sequence counters and the VIP streak. Completed history can remain after its game is deleted from inventory.

Source basis: Main; Queue2Play initialization and menu methods; RentalDataStore load/save methods; AppStorage.commit. Current method names are used because earlier line references changed after the rendering fixes.

02 / Current functionality

Area	Behavior in the current system
Login	Username or email, trimmed identifier, case-insensitive account matching, case-sensitive password. Empty fields have specific errors; unsuccessful authentication has one generic message.
Attempts	Three failed attempts trigger a visible 30-second cooldown. Failures carry across identifiers/menu returns in the running application. Successful login clears the counter.
Registration	Full name: 5-60 characters, letters/spaces/dots, at least two letters and a leading letter. Username: 3-20 permitted characters with at least one letter. Email: valid format, maximum 100 characters.
Password	8-20 characters. Letters-only, digits-only or mixed passwords accepted; symbols also accepted. Spaces, control characters and the file separator are rejected. Confirmation must match.
Browse / search / sort	All games or a selected platform. Exact ID, partial title or full platform name, ignoring case. T cycles default, title A-Z/Z-A and price low-high/high-low. Search/sort reset page one; X keeps the sort when clearing search.
Orders	Repeated cart selections combine. Queued plus cart quantity is limited to 3 copies per customer/game. Active rentals of the game block new ordering. A new cart line must fit current available stock.
Duration / pricing	1, 3, 7 or custom 1-30 days. Price uses the three-day rate; VIP receives 10% off. Unit cost is rounded to cents before multiplying by quantity.
Submission / checkout	One queued request per distinct cart game; no stock deduction on submission. Different requests can collectively exceed stock. Approval deducts stock, creates the active rental/history and saves.
Priority	VIP-first eligible requests, arrival order within each membership group. After three completed VIP approvals, an eligible regular gets a turn. Blocked requests wait while later eligible requests proceed.
Returns / inventory	Earliest due date first; single or comma-separated current-page rows. Restore full quantity; late fee is $1 per copy per started overdue day. Active rentals prevent game deletion; deleting a game cancels pending requests.
Large transaction screens	Receipts/results use N/P. Large batch previews use F/P so N/NO still declines. Records wrap and pages reserve space for choices and the prompt. Only usable page keys appear.

Older saved records can retain up to 5 copies per game; new ordering remains limited to 3. Requests do not reserve stock. Search/sort use display copies, and queue predictions consume copied stock only.

03 / DSA used and why

n is the number of examined records. A complete transaction can include several scans, validations and file writes.

DSA	Use and reason	Cost / detail
LinkedHashMap	Inventory by ID and cart by game; combines selections while retaining original order.	Average O(1) lookup/update; listing O(n). Java handles hashing.
ArrayList	Accounts, rentals, search results, display copies, session entries and page boundaries; simple indexed traversal.	O(1) indexed access; O(n) scan or middle removal.
Doubly linked queue	Previous/next nodes preserve VIP grouping and regular arrival order, without shifting a list on removal.	Regular append O(1); VIP insertion O(n); lookup O(n), then unlink O(1).
PriorityQueue / min-heap	Active rentals by earliest due date; rental number breaks ties. Polling a copied heap produces a sorted view.	Insert/poll O(log n); display O(n log n); selected removal searches O(n).
Stack	Newest customer history and administrator audit entries on top; displays do not delete stored history.	Push amortized O(1); display O(n). This is history, not undo.
HashSet	Duplicate identifier, queue/rental number and inconsistent saved-state checks.	Average O(1) membership/add.
Linear search	Exact game ID, title substring and platform search; account/request lookups also scan.	O(n) item checks; text matching also depends on text length. No pre-sort required.
Stable insertion sort	Catalog title/price ordering on a copy; restores queue arrival and rental-number ordering on load.	O(n squared) worst case, O(n) best case; equal catalog keys retain prior order.
Arrays / simulation	Return selection flags reject duplicates; snapshots restore state; queue simulation predicts copied-stock readiness.	Selection O(n); repeated queue scans make prediction O(n squared) worst case, with O(n) extra space.

Why these choices fit

They make ordering and validation understandable in a small console project. Large datasets would expose the cost of repeated queue previews, insertion sorting and full-state validation. No manually implemented tree, graph, binary search or jump search was found.

Source basis: Queue2Play fields/return selection; CheckoutQueue node and prediction methods; GameCatalog.linearSearch and insertion sorting; Rental.compareTo; User.rentalHistory; RentalDataStore validation/snapshot methods.

04 / F1 and F2 - retained intentionally

The owner explicitly confirmed both behaviors are intentional. These entries preserve the original review findings and their effects; they are not pending implementation requests.

F1 / Original priority: HIGH

Saved built-in passwords are restored at startup. AccountManager.ensureEmbeddedAccount checks admin, alex and juan against the code defaults. If a saved hash does not match the default password, it replaces that hash. Startup saves the change.

Earlier reproduction: A different admin password hash was saved in an isolated fixture, then accounts were loaded in a new system. The changed password failed and the original default matched again. No real account was changed for that probe.

Effect: An imported or externally changed built-in password does not survive startup. The application has no password-change screen. This policy applies to built-in accounts; normally registered accounts are still saved and reloaded.

Owner note: This default-restoration policy is intentional and remains unchanged. The earlier proposal to preserve changed built-in passwords is not part of the requested fixes.

Evidence: AccountManager.ensureEmbeddedAccount; Queue2Play startup. Earlier probe: changed password rejected=true, original default accepted=true. The source still contains this behavior; the earlier probe was not rerun for this document update.

F2 / Original priority: HIGH

The first legacy backup can retain plaintext passwords. Loading an old plaintext account hashes its password in memory. On the first save, AppStorage.commit copies the original file into accounts.txt.bak before replacing the live file. The first pre-upgrade backup is retained.

Earlier reproduction: A disposable legacy account was loaded and saved. The live file held a hash, while the backup still held the known test password.

Effect: Hashing the live account file does not remove plaintext from that original backup. New accounts use salted hashing. The documented consequence concerns legacy records and their retained backup.

Owner note: Keeping this original backup is intentional and remains unchanged. The earlier migration-backup correction is not part of the requested fixes. The backup observation is retained in the overview rather than removed.

Evidence: AccountManager.loadAccounts, User constructor, PasswordSecurity and AppStorage.commit. Earlier probe: live file hashed=true; backup keeps plaintext=true. Source behavior remains unchanged.

05 / F3 and F4 - fixes completed

F3 / Large transaction pages - FIXED

Earlier behavior: Receipts, batch previews and processed returns printed all entries at once. The original isolated measurements at height 36 / width 81 were:

Original view	Original fixture	Original rendered rows
Receipt plus navigation	40 session entries	57
Batch plus confirmation	40 planned requests	50
Return results plus navigation	9 returned rentals	106

Current behavior: TerminalUI calculates pages from complete rendered record heights and the configured viewport. It wraps long records and reserves room for navigation/prompt. Short or narrow screens use compact rows. Receipt and result pages use N/P; batch pages use F/P, preserving Y/YES and N/NO.

Each new page starts at the top. Repeated invalid result input clears before showing an error and the navigation panel, preventing accumulating panels. Small batches retain the existing checkout tape flow. Approvals still produce one slip each and the existing session status; no processing rules changed.

Verified: Forty receipt entries, requests and results were traversed at widths 120, 81, 60 and 40 and heights 50, 36, 30, 24 and 20. Every record stayed accessible and tested pages, options and prompts fit. Single-page results and repeated invalid keys were checked. A 12-request paged batch approved in predicted order and survived reload.

Methods: transactionPages, showTransactionPage, receiptPages, batchPages, returnResultPages and their navigation methods; Queue2Play checkout/return callers.

F4 / Shared-stock receipt waiting count - FIXED

Earlier behavior: Stock 3 with two requests for 2 copies each gave 1 ready / 1 waiting in the predicted queue, but 0 waiting in the receipt. The receipt checked requests independently against the same stock.

Current behavior: Receipt waiting = queued requests minus previewReadyRequests().size(). Receipt, administrator queue and customer positions therefore use the same copied-stock prediction.

Verified: Shared shortages, VIP fairness, approval, returns, restocking, zero stock and empty queues. Rendering leaves real stock, request order, session counters and VIP streak unchanged.

Methods: TerminalUI.receiptSummary and CheckoutQueue.previewReadyRequests. This fixes the display count without changing stock eligibility.

06 / Verification and limits

The following checks were executed again for this documentation update on 11 October 2026, using disposable data only.

Executed check	Observed result
Compile all top-level Java sources and BehaviorRegressionTest	Passed with Java JDK 24.
Full behavior regression suite	PASS: 43,586 checks; accounts, catalog, orders, quantities, queue fairness, returns, saving/reload and navigation.
Colored layout suite	PASS: 42,835 checks; NO_COLOR temporarily removed for the test process.
ASCII / no-color layout suite	PASS: 42,833 checks.
Dedicated real cooldown	PASS: Real cooldown lasted 30 seconds, including fast cosmetic mode.
Supplementary documentation fixtures	PASS: 16 assertions for stock add/decrease/reload, deletion, VIP refresh/reload, active/overdue lists, history and audit order.
Large-page coverage in the suites	All forty records accessible; page/prompt width and height bounds; usable keys; complete titles/totals; repeated errors; Back/Dashboard.
Paged batch integration in the full suite	12 requests approved in predicted order, stock deducted once, and all approvals saved/reloaded.
Receipt readiness coverage in the full suite	Predicted waiting counts agree across receipt and admin/customer queue displays; underlying state preserved.

Original review evidence

The initial review passed 1,801 full checks and 1,100 colored / 1,098 ASCII layout checks. Its separate probes reproduced F1-F4 and verified injected-save rollback. These are historical results; F3/F4 are now fixed and the latest results above supersede their earlier presentation status.

Storage observation

Earlier isolated runs sometimes encountered Windows AccessDeniedException/file-in-use during replacements. Later isolated runs succeeded. The latest full suite ran outside the sandbox to avoid the previously encountered test-environment file-lock problem. No storage behavior was changed. The cause of the intermittent contention was not established.

Not verified

Live Windows password masking/key capture; actual terminal color, resizing and scrollback appearance; real payment collection; power loss; long-term multi-process stress; every malformed record. Scripted output captures check dimensions, not screenshots of the running application.

No production data was loaded or edited by these tests. A successful test suite does not imply that every possible runtime or operating-system condition has been exercised.

07 / Current status and running checks

Item	Disposition	Evidence / next step
F1: built-in default restoration	Intentional; retained	Owner-confirmed policy; original finding and source explanation preserved.
F2: original legacy backup	Intentional; retained	Owner-confirmed policy; original backup observation preserved.
F4: predicted receipt counts	Completed	Shared-stock waiting counts agree with administrator/customer predictions.
F3: paged transaction views	Completed	All records accessible; tested navigation and prompt fit the configured viewport.
Windows file contention	Separate observation	If revisited, reproduce contention and check failed-save recovery independently.

How the checks were run

Following tests/README.md, sources and the test class were compiled into a disposable build folder. The installed JDK executables were used. **The production application was not launched against the real data directory.**

```
javac -encoding UTF-8 -d <isolated-build>
  <all top-level Java sources> tests/BehaviorRegressionTest.java

java "-Djava.io.tmpdir=<isolated-build>"
  -cp <isolated-build> BehaviorRegressionTest
```

Additional runs: layout and cooldown.
ASCII layout: -Dq2p.ascii=true -Dq2p.color=false.
Colored layout: NO_COLOR removed temporarily; -Dq2p.color=true.

Running the system normally

Use .\run.ps1 from the project folder in PowerShell. It compiles the application and reads the terminal dimensions. Without the launcher, Java uses configured q2p.width/q2p.height, COLUMNS/LINES, or defaults of 120 columns / 50 rows. Pagination keys appear only when usable. Restart after changing the terminal size so the launcher supplies the new dimensions.

Sharing the repository

Generated build/test output can be excluded. Decide whether to publish tests and documentation. Review account/log data and retained backups before sharing. Ignoring a path does not untrack files already committed; deleting files now does not erase older Git history. No repository cleanup was performed for these documents.

Review basis

Current renamed Java sources, run.ps1, the beginner Java skill, tests/README.md, BehaviorRegressionTest, and the original program review. F1/F2 remain intentional and unchanged; F3/F4 are recorded as implemented and verified. No new program changes were made for this documentation request.